WEEK 1 · DAY 1 · OVERVIEW

The ML Workflow and Simple Regression

From your first line of code to your own AI agents.
Every lesson is a fixed income problem.



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

A credit team has taken on a benchmark of 821 corporate bonds from 150 issuers, rated AAA to CCC.

It wants to know how much of the difference in spread between bonds one characteristic can account for: the rating, the duration, or the coupon.

The answer has to hold on bonds the model has never seen, because those are the only bonds a model is ever asked about.

The bonds and issuers are invented for this course.

OBJECTIVES

Explore the file, fit a simple linear regression of OAS on each candidate, score each model in bp on bonds it was not fitted on, and find the single characteristic that says the most about spread.

ABOUT 45 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

Your Route Through Day 1

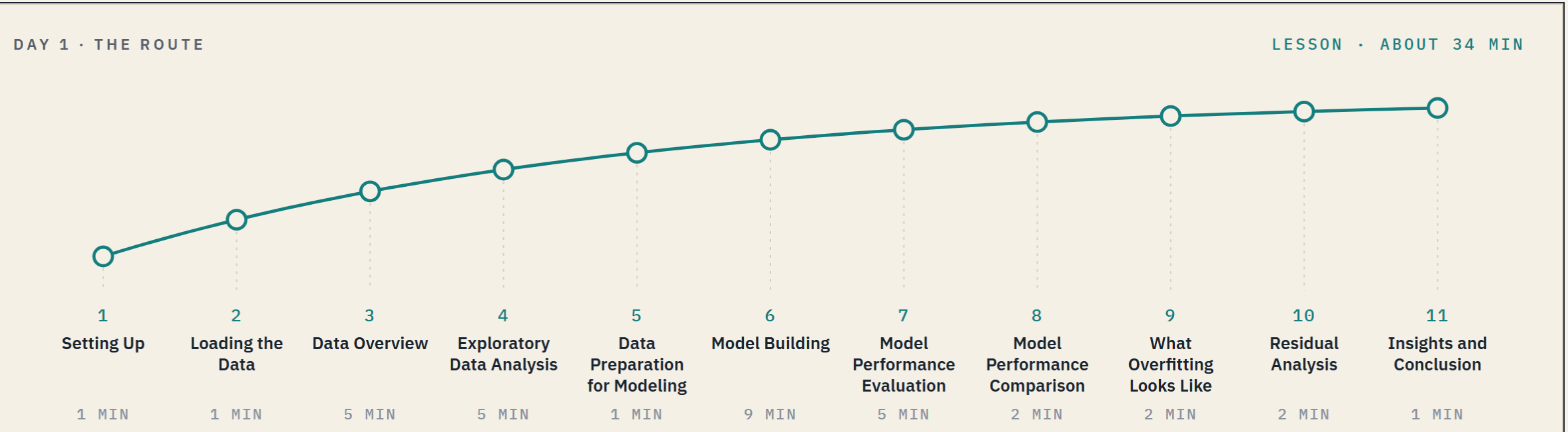


FIGURE 1 Eleven sections, in order, with about how long each takes.

Time. About 34 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SEED AND THE DATA

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
```

THE LIBRARIES

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
sns.set_theme(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 5)
pd.set_option("display.float_format", "{:,.3f}".format)
```

Run this cell and move on: it fixes the seed and finds the course data.

Read the bonds and score each rating.

Loading the Data

READ THE FILES

```
bonds = pd.read_csv(DATA + "benchmark.csv")
ratings = pd.read_csv(DATA + "ratings.csv")
```

SCORE EACH RATING

```
data = bonds.merge(ratings[["rating", "score", "grade"]], on="rating")
```

THE RATING SCALE

ratings.csv adds **score** (1 for AAA up to 18 for CCC, one step per notch) and **grade** (Investment Grade or High Yield).

THE COLUMNS IT USES

- **cusip**: the bond's identifier
- **issuer**: the issuer's name, with **ticker** its short code
- **rating**: credit rating, AAA to CCC
- **coupon**: annual coupon, % of par
- **price**: clean price, per 100 of par
- **yield_pct**: yield to maturity, % (a Treasury yield plus the OAS)
- **oas**: option-adjusted spread, bp, **the target**
- **duration**: modified duration, years
- **dts**: spread duration times OAS
- **amount_outstanding**: face value outstanding, dollars

Data Overview

A SUMMARY OF THE NUMBERS

	MEAN	50%	MAX
coupon	5.852	5.625	11.000
price	99.466	99.637	127.511
accrued	1.502	1.373	5.159
yield_pct	5.985	5.493	25.140
oas	179.079	125.000	2,074.000
duration	5.981	5.919	16.042
spread_duration	5.981	5.919	16.042

SHAPE AND GAPS

```
data.shape
```

```
(821, 21)
```

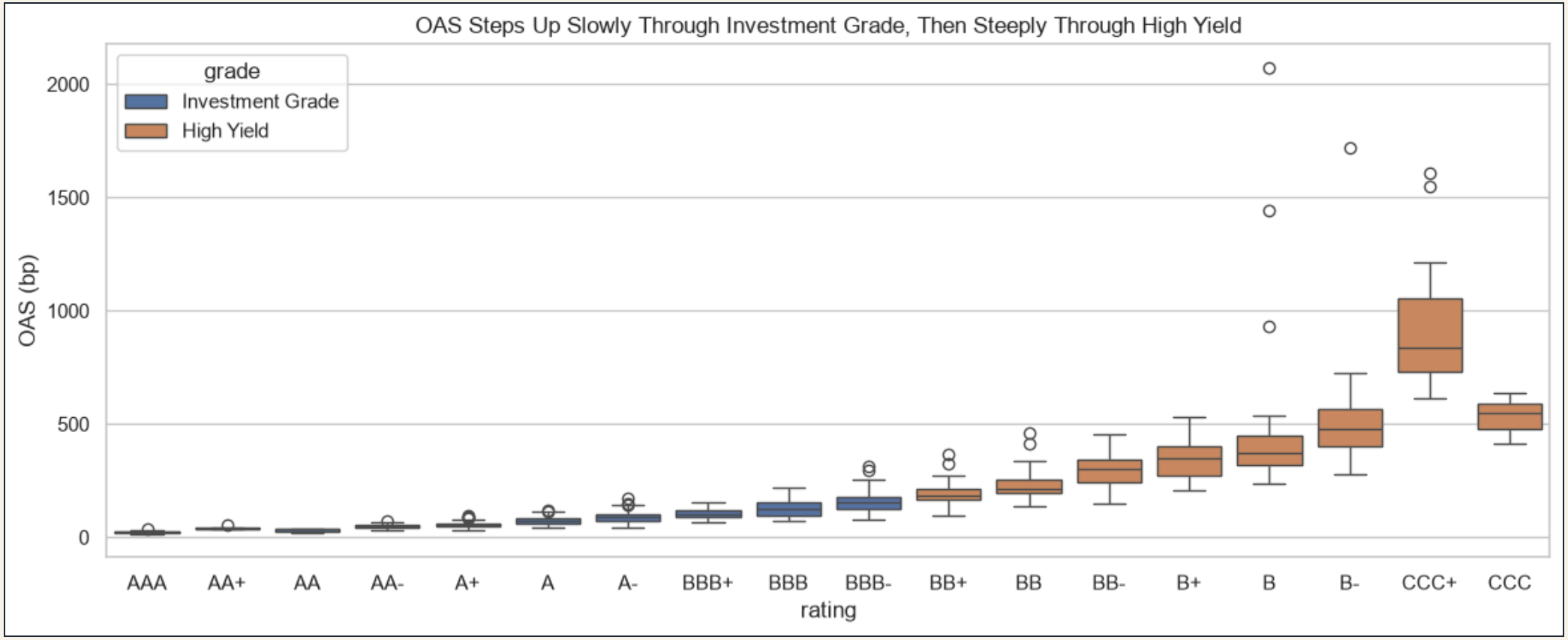
DUPLICATES

```
print(data["cusip"].duplicated().sum())
```

```
0
```

The file is complete and clean, so we can go straight to exploring it.

Exploratory Data Analysis



A straight line will struggle at both ends of this curve. We fit one anyway, because it is the simplest model there is, and measure how far it misses.

Data Preparation for Modeling

THE ROWS IT WORKS FROM

	SCORE	DURATION	COUPON	OAS
0	9	6.000	4.875	83
1	9	6.834	5.750	122
2	9	9.848	5.000	123

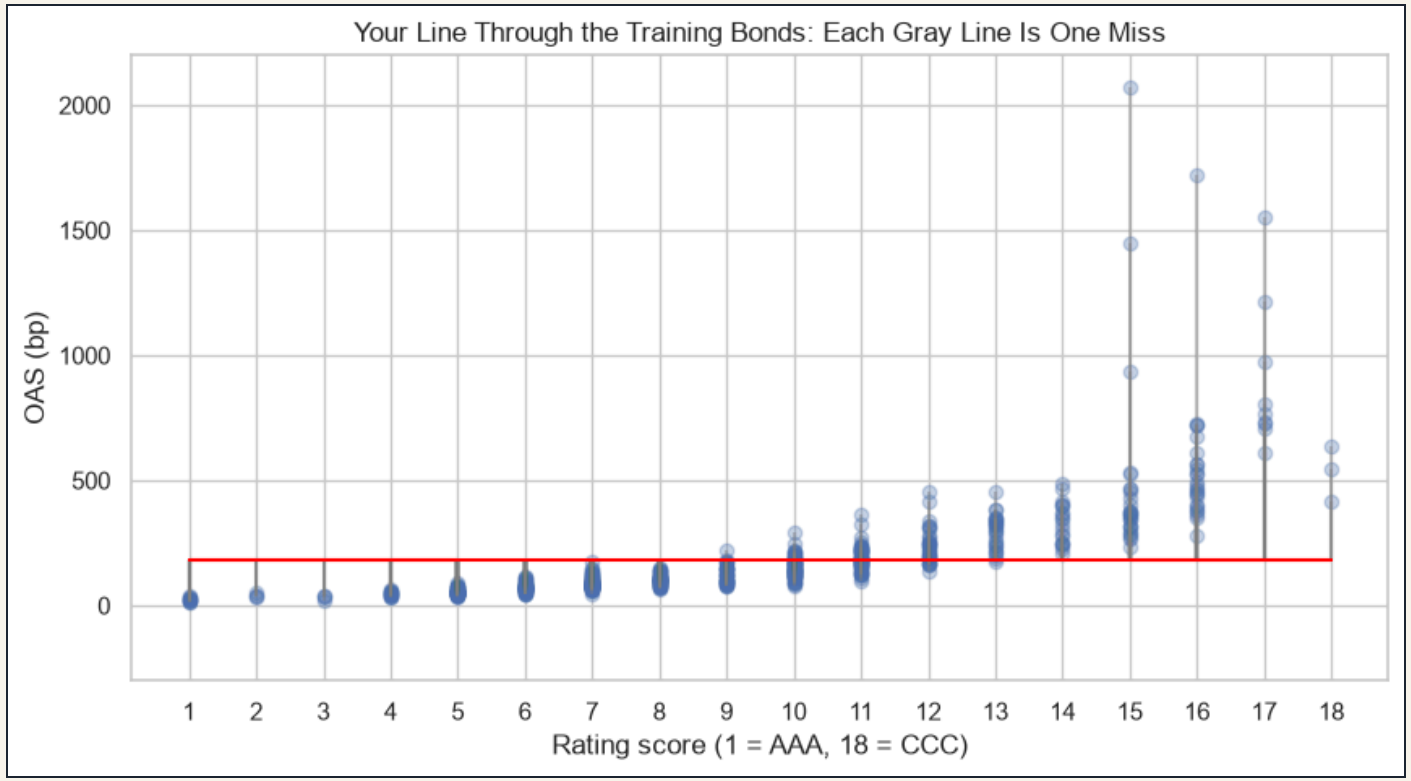
FEATURES, TARGET, AND THE SPLIT

```
X = data[["score", "duration", "coupon"]]
y = data["oas"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=SEED)
print("Training bonds:", len(X_train), " Test bonds:", len(X_test))
```

```
Training bonds: 615  Test bonds: 206
```

The test bonds stay untouched until the models are scored: that score is the only honest one.

Model Building



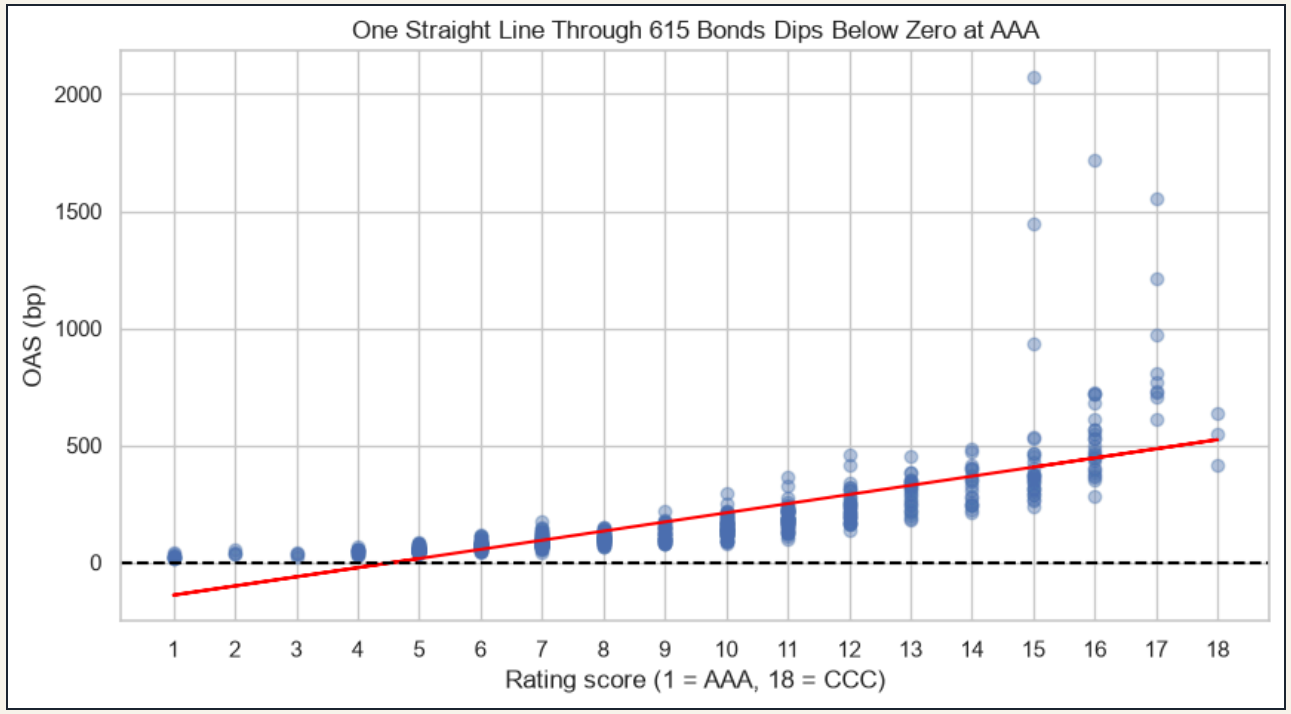
WHAT IT PRINTED

```
draw_line(slope=0.0, intercept=180.0)

MAE:  115.1 bp
RMSE: 192.6 bp
```

The sliders start at a flat line at 180 bp, about the average spread: MAE 115.1 bp, RMSE 192.6 bp.

The Line the Computer Fits



THE EQUATION

```
OAS = ( 39.05 ) * score + ( -179.06 )
```

R-SQUARED

```
model_score.score(X_train[features_score], y_train)
```

```
0.5227499437906944
```

Rating gives a workable line. Next we ask whether duration does as well.

Model Performance Evaluation

FIVE MEASURES

- **MAE** (bp): the average length of a gray line.
- **RMSE** (bp): square each miss, average, take the root, so long misses count far more.
- **MAPE** (%): each miss as a percent of the bond's own spread, averaged.
- **R-squared**: the share of the variation in spread the model accounts for.
- **Adj. R-squared**: R-squared with a small charge for each feature. With one feature it sits almost on top of R-squared; it starts to matter on Day 2, when features are added.

TRAINING SCORE

```
score_train_perf = model_performance_regression(model_score, X_train[features_score], y_train)
score_train_perf
```

	RMSE	MAE	MAPE	R-SQUARED	ADJ. R-SQUARED
0	133.046	65.624	56.365	0.523	0.522

Each model has a score on unseen bonds. Next, line them up side by side.

Model Performance Comparison

TWO OF THE THREE LINES, ON TRAINING BONDS

	OAS VS RATING SCORE	OAS VS COUPON
RMSE	133.046	135.597
MAE	65.624	77.223
MAPE	56.365	66.227
R-squared	0.523	0.504
Adj. R-squared	0.522	0.503

HOW EACH TABLE IS BUILT

```
models_train_comp = pd.concat([score_train_perf.T, duration_train_perf.T, coupon_train_perf.T], axis=1)
models_test_comp = pd.concat([score_test_perf.T, duration_test_perf.T, coupon_test_perf.T], axis=1)
```

What Overfitting Looks Like

THE IDEA

The memoriser writes down every training bond's spread.

Asked about a bond it has seen, it reads its notes.

THE MEMORISER

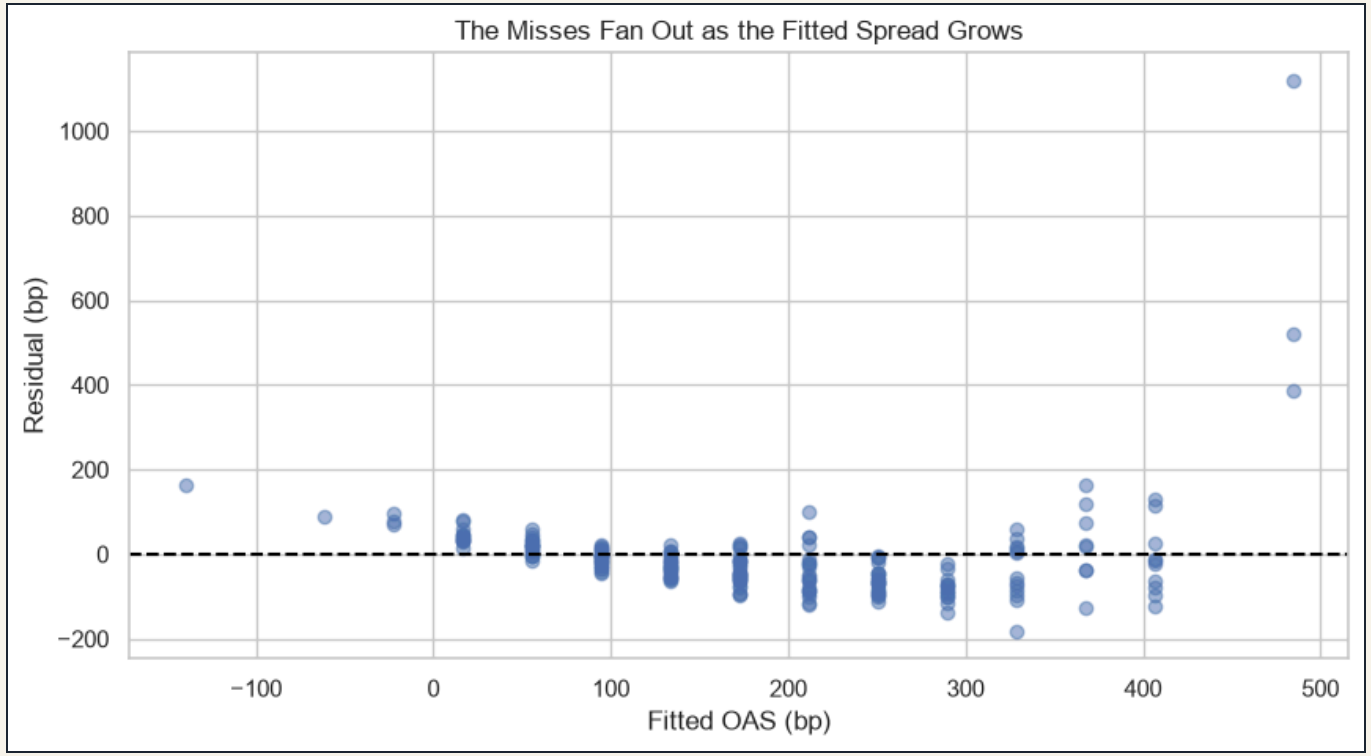
```
notes = dict(zip(data.loc[X_train.index, "cusip"], y_train))
memo_train = data.loc[X_train.index, "cusip"].map(notes).fillna(y_train.mean())
memo_test = data.loc[X_test.index, "cusip"].map(notes).fillna(y_train.mean())
memo_train_mae = mean_absolute_error(y_train, memo_train)
print("Training MAE:", round(memo_train_mae, 1), "bp")
```

```
Training MAE: 0.0 bp
```

THEN ON NEW BONDS

```
memo_test_mae = mean_absolute_error(y_test, memo_test)
print("Test MAE:", round(memo_test_mae, 1), "bp")
```

Residual Analysis

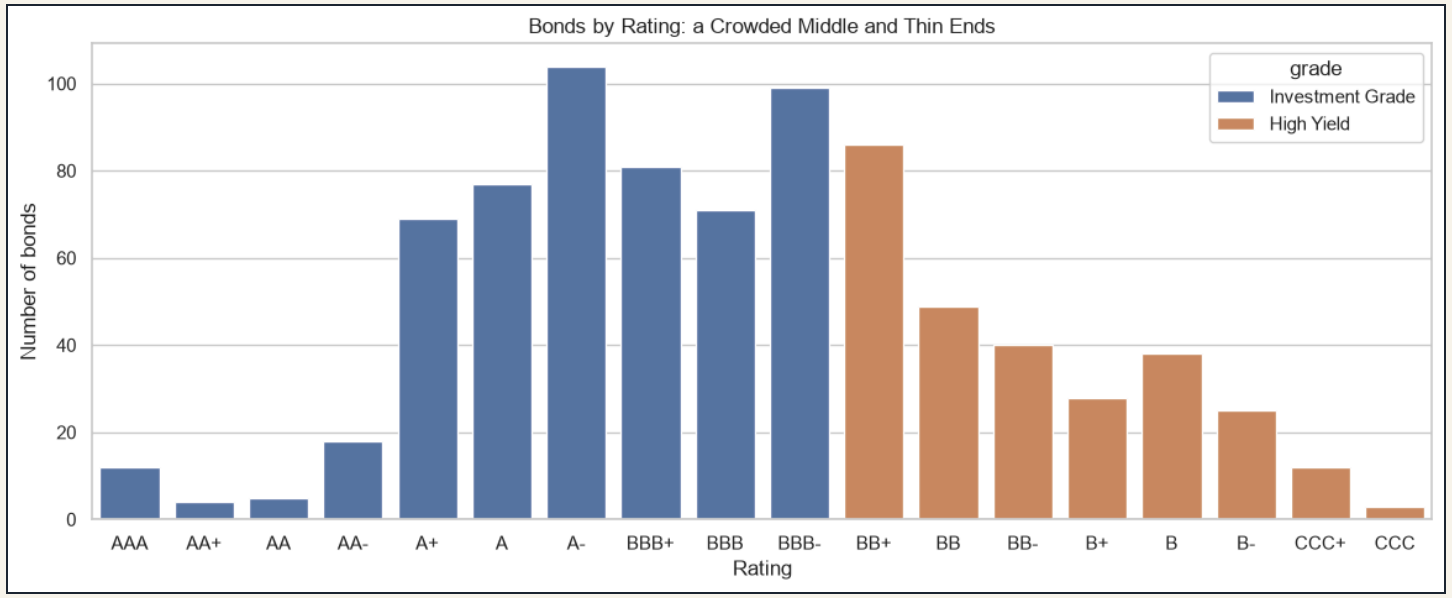


THE LARGEST MISSES

	ISSUER	RATING	OAS	RESIDUAL
196	Praxiskan Finance	CCC+	1605	1,120.200
199	Praxiskan Finance	CCC+	1004	519.200
198	Praxiskan Finance	CCC+	871	386.200
393	Wyxundra Energy	B+	532	164.300
155	Dralane Networks	AAA	23	163.000

The misses grow with the spread, and the line has the wrong shape at both ends: Day 2 gives every rating its own step, and Day 4 bends the line with log spreads.

Insights and Conclusion



THE SHAPE OF THE LINE

The rating line is a straight line on a curve: below zero at AAA, short of the widest high yield bonds.

THE TEST BONDS

203 of the 206 test bonds share an issuer with a training bond, so "never seen" is weaker than it sounds; Day 5 splits by issuer.

One straight line on rating carries about half the story; Day 2 gives every rating its own step.

You Can Now

- Frame a modeling task and run a data overview.
- Read univariate and bivariate charts, and spot a feature built from the target.
- Fit a simple linear regression, write its equation, and say what its slope and R-squared mean.
- Score and compare models on training and test bonds with five measures.
- Read a residual plot, and tell a model that has learned from one that has memorised.

NEXT
WEEK 1 · DAY 2

Multiple Regression and Dummy Variables

Day 2 gives every rating its own step.